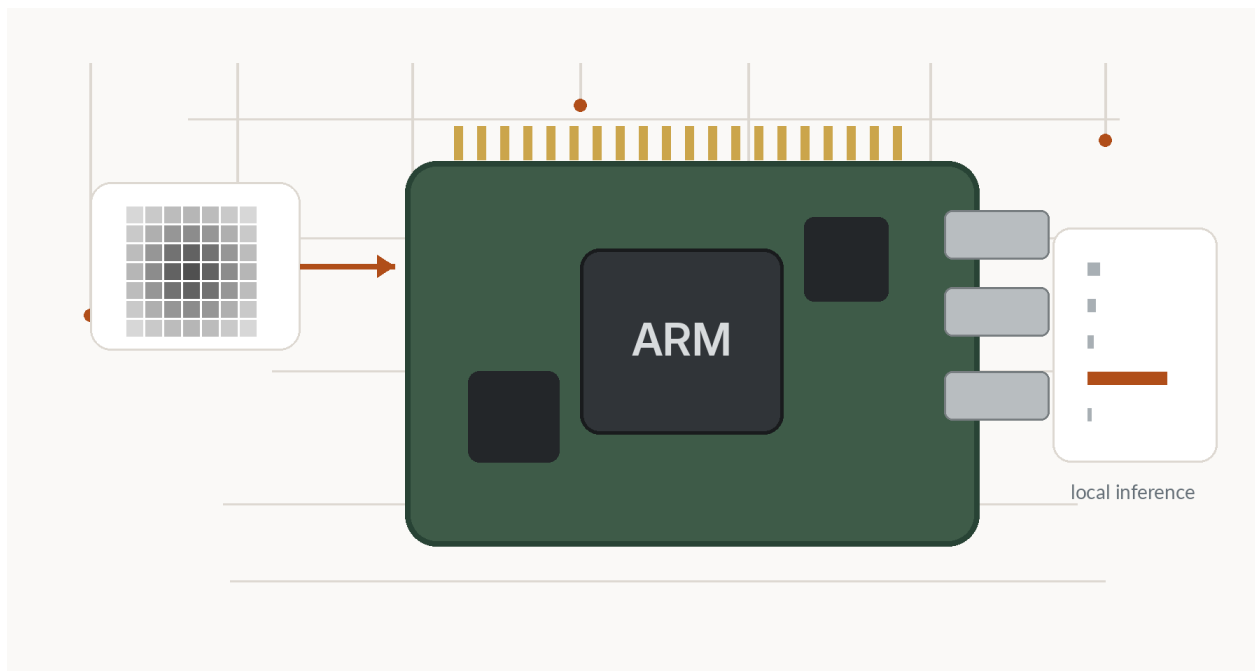


POSTGRADUATE PROGRAMME IN EMBEDDED SYSTEMS

Artificial Intelligence in Embedded Systems

PRACTICAL LABORATORY 03

Deploy and compare trained models on the Raspberry Pi



Tiago Fonseca | ISEP | Student handout | v3

LAB PURPOSE

Run a supplied catalogue of trained neural networks, verify their predictions and measure how architecture and numerical representation affect storage, memory and inference time on the Raspberry Pi.

1

Mission and working route

Continue the deployment work after the PL2 Colab session.

1.1 Starting point

In PL2 you completed the notebook work in Google Colab. PL3 begins with trained models and focuses on the Raspberry Pi stage. You do not need to finish the old PL2 deployment instructions before starting this handout. The supplied PL3 package contains its own models, test images and Python tool.

The networks use the same Fashion-MNIST task and preprocessing as PL2. They are new lecturer-trained reference models, not copies of your group's weights. Their predictions and accuracy may therefore differ from your notebook. Keep your PL2 notebook as evidence of your own training work.

START HERE Read Chapter 2, then follow Chapter 3 on the Raspberry Pi. Download PL3_Model_Pack_v3.zip from Moodle and create the dedicated ~/PL3 folder and Python environment. This laboratory is for practice and discussion; no upload or formal submission is required.

1.2 What you will establish

Question	Evidence to collect
Did the intended artifact arrive?	Model and dataset SHA-256 checks.
Does inference work correctly?	Input/output contract, labelled examples and accuracy.
What changes with width or depth?	Parameter counts, file size and measured execution cost.
What changes with numerical format?	FP32, FP16 and INT8 from the same trained checkpoint.
What can we conclude on this board?	A comparison using one declared, repeated protocol.

1.3 Working method and prerequisites

Work in pairs. Predict the result, execute one labelled command block, note your observations and discuss them before continuing. Keep the same board and power supply throughout the comparison. You need a Raspberry Pi with 64-bit Raspberry Pi OS, a desktop, internet access and permission to install the Python environment tools. Chapter 3 provides the complete setup.

Use the Pi browser to download the materials and its terminal for every RASPBERRY PI block. All files and Python packages for this laboratory stay in ~/PL3. REFERENCE ONLY blocks explain how the supplied artifacts were produced; do not execute those on the Pi. Commands save results locally and refuse to overwrite an existing run.

2

Background for PL3

Understand the artifacts and measurements before executing them.

2.1 A named catalogue of neural networks

The name d1w32 means one hidden Dense layer with 32 neurons. Depth counts only hidden layers. Every model also has a Flatten operation and a final Dense layer with ten Softmax outputs. ReLU follows each hidden Dense layer. For example, d2w64 contains two hidden layers of 64 neurons each, followed by the ten-class output layer.

Architecture	Hidden layers	Parameters	Dense MACs
d1w32	32	25,450	25,408
d1w64	64	50,890	50,816
d1w128	128	101,770	101,632
d2w64	64 + 64	55,050	54,912
d3w64	64 + 64 + 64	59,210	59,008

The suffix identifies the stored numerical representation: d1w64_fp32.tflite, d1w64_fp16.tflite and d1w64_int8.tflite are three exports of the same selected d1w64 checkpoint. The remaining four architectures are supplied in FP32. There are seven .tflite files in total.

2.2 How the reference models were trained

The lecturer trained each architecture from fresh weights using the PL2 split: 54,000 training images and 6,000 validation images from the official training set, with seed 42. Each run used Adam with learning rate 0.001, batches of 128 and 30 epochs. The checkpoint with the lowest validation loss was retained independently for each architecture.

The official 10,000 test images were evaluated after these choices were frozen. The test results did not select the architectures or checkpoints. One seeded run per architecture supports a classroom comparison, but does not establish a universal ranking. Equal epochs mean equal passes through the data, not equal training time.

CONTROLLED COMPARISON For width, compare d1w32, d1w64 and d1w128 in FP32. For depth, compare d1w64, d2w64 and d3w64 in FP32. For quantization, hold d1w64 fixed and compare its three formats.

2.3 How the trained models became LiteRT artifacts

TensorFlow/Keras represents the network during training and updates its parameters. Saving a .keras checkpoint preserves a development model. Conversion then produces a .tflite file containing the inference graph and learned constants that the LiteRT runtime can execute. Conversion does not train the network again.

For the FP32 artifact, the exporter loads the selected checkpoint and asks the converter to produce the default floating-point representation. Pixel normalization stays outside the model, as in PL2. The output file is written as binary data.

REFERENCE ONLY · FP32 export on the training computer

```
from pathlib import Path
import tensorflow as tf
from tensorflow import keras

model = keras.models.load_model("dlw64.keras")
converter = tf.lite.TFLiteConverter.from_keras_model(model)
artifact = converter.convert()
Path("dlw64_fp32.tflite").write_bytes(artifact)
```

The inference artifact does not require the optimizer or the training loop. On the Pi, Python loads LiteRT, creates an Interpreter, allocates its tensors, places one input in the input tensor and invokes the model. The trained weights stay fixed throughout this laboratory.

Training computer	Raspberry Pi
Load data and update weights with TensorFlow/Keras.	Load the supplied .tflite artifact with LiteRT.
Choose a checkpoint using validation loss.	Inspect the input and output tensor contract.
Convert and validate every exported representation.	Encode inputs, invoke the model and interpret outputs.
Save artifacts, hashes and reference outputs.	Measure quality and execution cost on the target.

CONVERSION NEEDS A CHECK Producing a file is only the first step. The exported models were executed locally and compared with Keras. On the Pi, you will check artifact identity, tensor metadata and reference predictions again.

2.4 FP32 FP16 and INT8 in this package

FP32 is the reference export. FP16 reduces the precision used to store floating-point weights, while this exported model keeps Float32 input and output tensors. Its CPU execution can expand weights or use different kernels; a smaller file alone does not prove lower RAM or latency.

REFERENCE ONLY · FP16 export from the same checkpoint

```
converter = tf.lite.TFLiteConverter.from_keras_model(model)
converter.optimizations = [tf.lite.Optimize.DEFAULT]
converter.target_spec.supported_types = [tf.float16]
Path("dlw64_fp16.tflite").write_bytes(converter.convert())
```

The INT8 export uses 500 fixed, randomly selected training images to calibrate activation ranges. These examples use exactly the deployed normalization. Calibration estimates numerical ranges; it does not update weights using gradient descent. The validation and test sets are excluded from this calibration subset.

REFERENCE ONLY · Integer export with calibration

```
def representative_data():
    for i in calibration_indices:
        yield [x_train[i:i + 1]] # Float32, already / 255.

converter = tf.lite.TFLiteConverter.from_keras_model(model)
converter.optimizations = [tf.lite.Optimize.DEFAULT]
converter.representative_dataset = representative_data
converter.target_spec.supported_ops = [
    tf.lite.OpsSet.TFLITE_BUILTINS_INT8
]
converter.inference_input_type = tf.int8
converter.inference_output_type = tf.int8
Path("dlw64_int8.tflite").write_bytes(converter.convert())
```

This configuration requires supported integer operations and uses INT8 input and output interfaces. Biases and intermediate accumulations can use wider integer types. The FP16 and INT8 exports share the FP32 checkpoint; they were not retrained separately. Quantization may change scores or the winning class, so accuracy must be measured again.

REFERENCE CODE model, x_train and calibration_indices belong to the lecturer's training recipe. These snippets explain the export; they are not additional setup tasks for students. All seven finished files are already in the Moodle package.

2.5 The tensor contract and preprocessing

The stored test images are uint8 arrays with shape (28, 28) and pixel values from 0 to 255. For each image, the tool adds a batch dimension and divides by 255 once. The resulting real input has shape (1, 28, 28). Flatten is inside the network: do not reshape the image into (1, 784) before feeding these artifacts.

Artifact	Input	Output
FP32	Float32 (1, 28, 28)	Float32 (1, 10)
FP16 weights	Float32 (1, 28, 28)	Float32 (1, 10)
Full integer export	INT8 (1, 28, 28)	INT8 (1, 10)

For an integer input, the normalized real value must be encoded with that tensor's scale and zero-point. The tool rounds to the nearest integer and clips to the INT8 range. Casting a normalized number directly to int8 would discard most of its information.

REFERENCE ONLY · The encoding implemented in lab.py

```
real = image[None].astype(np.float32) / 255.0
scale, zero_point = input_info["quantization"]
encoded = np.clip(
    np rint(real / scale) + zero_point, -128, 127
).astype(np.int8)
```

In the supplied INT8 artifact the input scale is approximately 1/255 and the zero-point is -128. These are properties of this file, not values to hard-code for every model. The command in Chapter 3 reads and prints the actual metadata.

Raw pixel	Normalized real value	Input INT8 code
0	0	-128
128	approximately 0.50196	0
255	1	127

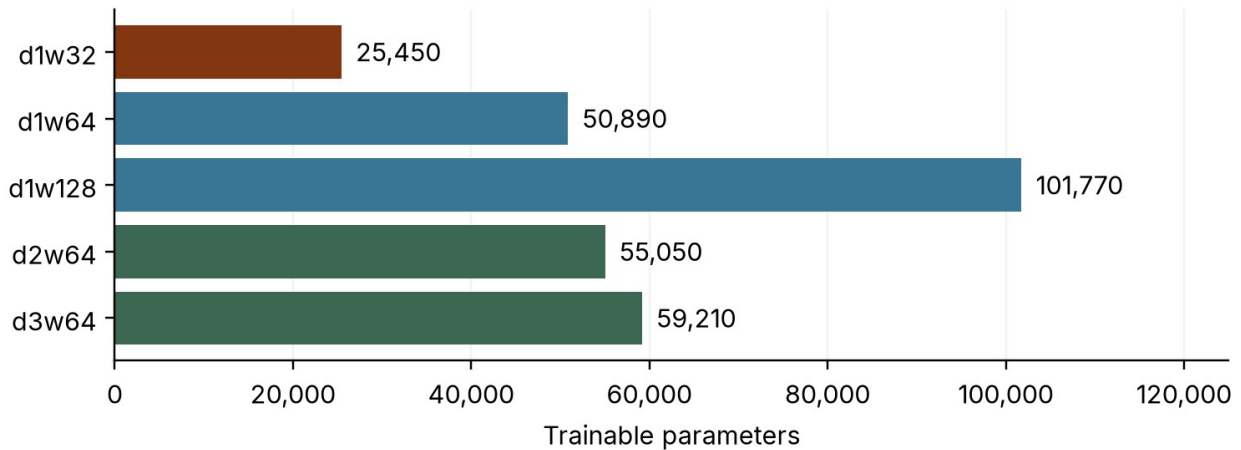
An integer output uses its own scale and zero-point. Reconstruct a real score as (integer code - zero-point) × scale. The ten outputs follow the Fashion-MNIST class order. Argmax chooses the largest score, but a high score does not guarantee that the predicted class is correct.

PREDICT Why would the FP16 artifact fail if we fed a Float16 input? Why can two tensors in the same INT8 model have different scales? Answer from the tensor contract, not from the filename.

2.6 Parameters computation and memory

A Dense layer with n inputs and m outputs stores $n \times m$ weights and m biases. It performs $n \times m$ multiply-accumulate operations, or MACs, per example. The catalogue counts these Dense MACs only; it excludes activations, Softmax, tensor movement and runtime overhead.

For d1w64, the parameter count is $(784 + 1) \times 64 + (64 + 1) \times 10 = 50,890$. Its Dense MAC count is $784 \times 64 + 64 \times 10 = 50,816$. The four bytes per FP32 parameter give a storage estimate of 203,560 bytes, before graph information and file-format overhead.



Trainable parameter counts of the five supplied architectures. A wider first layer can add more parameters than another small hidden layer.

The .tflite file size measures persistent storage. Process RAM also includes the interpreter, libraries, accessible weights, intermediate activations and work buffers. The Linux RSS metric measures resident process memory, including shared pages. It is not the memory occupied exclusively by the neural network.

2.7 A measured inference time

The first invocation may include preparation that later calls reuse. The benchmark records that first call, performs 20 further warm-up calls and measures 500 invocations. It repeats the experiment three times, using a fresh process for every model and trial and varying the model order.

Only `Interpreter.invoke()` is timed. Input preparation, tensor copies, model loading and result interpretation sit outside that interval. This answers how long the model invocation takes in this setup; it does not measure the complete application response time. Median describes a typical observed call, while p95 describes the slower part of the sample. Neither provides a worst-case deadline guarantee.

3

Practical laboratory

Create a dedicated workspace and explore the models on the Raspberry Pi.

3.1 Obtain the official model package

On the Raspberry Pi desktop, open the browser, log in to Moodle and open the PL3 section of this course. Download PL3_Model_Pack_v3.zip to Downloads and wait until the download finishes. Keep this PDF open on the Pi while working through the commands.

A FRESH WORKSPACE FOR PL3 Use PL3_Model_Pack_v3.zip and open a new terminal on this Pi. The folder ~/PL3 will contain the models, data, Python tool, environment and results. Follow the setup below from the beginning; it is independent of the folders and environments used in previous laboratories.

The commands assume ~/Downloads/PL3_Model_Pack_v3.zip. Use the browser's Show in folder option to check the saved file. If the folder has another name, such as Transferências, replace Downloads in the commands. If the browser added (1) to the filename, rename the completed ZIP to the name above.

RASPBERRY PI · Identify the target

```
hostname
uname -m
python3 --version
```

A 64-bit Raspberry Pi OS normally reports aarch64. Record these details. Create a new ~/PL3 folder and extract the ZIP into it. If ~/PL3 already exists, stop and rename that previous folder before continuing. The && below runs extraction only when the new folder was created successfully.

RASPBERRY PI · Extract and enter the workspace

```
ls ~/Downloads/PL3_Model_Pack_v3.zip
mkdir ~/PL3 && \
python3 -m zipfile -e ~/Downloads/PL3_Model_Pack_v3.zip ~/PL3
cd ~/PL3
ls
ls models
```

CHECKPOINT You should see lab.py, requirements.txt, manifest.json, models and data directly inside ~/PL3, with seven .tflite files in models. Stay in this folder for the following steps. The experiments will create a results folder here automatically.

3.2 Create the dedicated PL3 environment

Prepare Python's virtual-environment support on Raspberry Pi OS. These commands need internet access and may ask for the Pi account password. A virtual environment keeps this laboratory's Python packages inside ~/PL3/.venv. Create it once, in the new workspace.

RASPBERRY PI · Prepare the operating system tools

```
sudo apt update
sudo apt install -y python3-venv
```

RASPBERRY PI · Create and activate the PL3 environment

```
cd ~/PL3
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --only-binary=:all: -r requirements.txt
which python
python -c "from ai_edge_litert.interpreter import Interpreter"
```

which python should end in /PL3/.venv/bin/python. The import check should finish without an error. This environment installs LiteRT and its dependencies; full TensorFlow/Keras is unnecessary. Keep this terminal open. If you reopen it later, use the short resume block in section 3.13.

3.3 Check identity and reference behaviour

The check command verifies the dataset hashes, all model hashes and tensor contracts. It then executes the first ten test images with each artifact and compares their scores with the supplied local reference. The report records versions and identifies the target. Small cross-platform rounding differences can occur; the tool uses an explicit score tolerance and reports changed classes separately.

RASPBERRY PI · Validate the package before measuring

```
python lab.py check --output device_check.json
python lab.py catalog
```

Expect seven PASS lines, followed by the environment and reference-comparison report. Inspect any REVIEW result or contract/hash error with the lecturer before proceeding. PASS verifies these checks, not every possible input. device_check.json stays in ~/PL3 for your own reference.

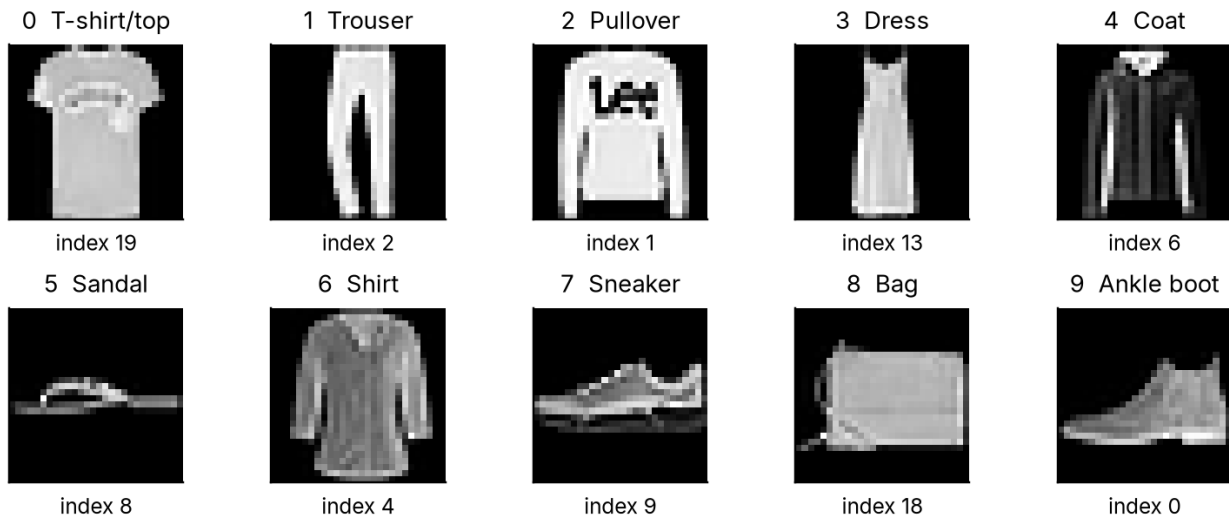
Record	Your result
Board and hostname	
CPU architecture and Python version	
LiteRT version and environment path	
Seven checks passed and differences inspected	

3.4 Inspect and run one trained model

Start with `d1w64_fp32`. The `inspect` command prints its exact file size, hash and tensor contract. The `predict` command loads a test image, applies the correct preprocessing, invokes LiteRT and prints the true label and the three highest output scores. The index is an image position, not a class number.

RASPBERRY PI · Inspect the contract and predict

```
python lab.py inspect --model d1w64_fp32
python lab.py predict --model d1w64_fp32 --index 0
python lab.py predict --model d1w64_fp32 --index 42
```



Examples from the supplied official test split. The image index identifies a sample; the class number identifies its label.

The input report should show batch size one and a normalized Float32 range. A correct run need not classify every image correctly. Compare the predicted class with the ground-truth label and distinguish a wrong prediction from an input-processing or runtime error.

Observation	Your explanation
Input shape and dtype	
True and predicted class for image 0	
True and predicted class for image 42	
What does the largest score tell you?	

INTERPRET Why would training for more epochs be irrelevant to this command? Identify which file contains the learned parameters and which operation applies them to the image.

3.5 Evaluate the same labelled examples

A single prediction checks the route through the application, but cannot estimate classification quality. Evaluate the five FP32 architectures on test indices 0 to 499. The subset is fixed in the original dataset order. It is a quick classroom comparison, not a newly sampled or stratified benchmark, and its accuracy may differ from the full 10,000-image result.

RASPBERRY PI · Evaluate the architecture catalogue

```
python lab.py evaluate --models architecture \
  --limit 500 --run architecture_quality
python lab.py report --run architecture_quality
```

The command writes accuracy.csv, one predictions CSV and one confusion matrix per model, plus protocol.json. Each predictions row records the image index, true label, predicted class and top score. Accuracy is correct predictions divided by the same 500 examples. One additional correct image changes this subset accuracy by 0.2 percentage points.

Architecture	Correct out of 500	Accuracy percent
d1w32		
d1w64		
d1w128		
d2w64		
d3w64		

reference_changed_classes compares the current board with the lecturer's local execution of the same artifact on up to the first 500 images. It is a deployment consistency check. It does not compare different architectures and is not the number of classification errors.

DISCUSS Do the widest and deepest models necessarily have the highest observed accuracy? Keep the test subset and preprocessing fixed while discussing capacity. Do not retrain or tune a model based on these test results and describe the reused test as an independent final check.

If instructed and time permits, repeat with --limit 10000 and a new run name. Always label which sample size produced a result. A comparison between one model on 500 images and another on 10,000 images would not use the same workload.

3.6 Compare width and inference latency

Predict how width might affect parameter storage and computation. At depth one, d1w32, d1w64 and d1w128 differ only in hidden width within the common training recipe. The selected checkpoints can come from different epochs because selection used each model's validation loss.

Run the architecture benchmark once. It measures all five FP32 models and saves the evidence used in sections 3.6 to 3.8. Keep other applications quiet and leave the power supply and cooling setup unchanged. Each model uses one requested inference thread and batch size one.

RASPBERRY PI · Measure all five architectures

```
python lab.py benchmark --models architecture \
  --warmup 20 --calls 500 --repeats 3 \
  --threads 1 --run architecture_cost
python lab.py report --run architecture_cost
```

The terminal reports every trial. summary.csv combines the three trials per model, while trials.csv and the individual JSON files retain the details and raw times. median_of_trial_medians_ms is the median of three trial medians. median_of_trial_p95_ms is the median of three separate p95 values, not a pooled percentile of 1,500 calls.

Width	File KiB	Median ms	p95 ms
32			
64			
128			

Read the result before explaining it

Do the file sizes follow the parameter counts? Does doubling hidden width double the measured latency? A Dense MAC count estimates arithmetic work, but kernels, vectorization, memory access and fixed overhead influence actual time. Report the observed ratio rather than assuming a proportional change.

Inspect min_trial_median_ms and max_trial_median_ms. If the ranges overlap substantially, a small difference in summary medians may not support a confident ranking. Keep all measurements in milliseconds and identify the board on which they were collected.

PREDICT THEN VERIFY Calculate the latency ratio d1w128 / d1w32 from your measurements. Compare it with the Dense MAC ratio. Explain why the two ratios need not match.

3.7 Compare depth at a fixed width

Reuse `architecture_quality` and `architecture_cost`. Compare d1w64, d2w64 and d3w64, keeping the hidden width at 64 and the numerical representation at FP32. Do not run the benchmark again just to obtain a more attractive ranking.

The first hidden layer consumes 784 image values, whereas an additional hidden layer consumes only 64 values. Each extra 64-to-64 hidden layer therefore adds $64 \times 64 + 64 = 4,160$ parameters and 4,096 Dense MACs. The output layer remains ten units.

Architecture	Parameters	Dense MACs	Accuracy percent
d1w64	50,890	50,816	
d2w64	55,050	54,912	
d3w64	59,210	59,008	

Architecture	Median ms	p95 ms	Peak process RSS MiB
d1w64			
d2w64			
d3w64			

Architecture and embedded cost

Extra layers introduce sequential stages and more intermediate values. They also introduce more nonlinear transformations, which can change the function the network learns. More depth can improve representation, but it can also make optimization harder or add cost without a useful quality gain.

Compare d1w128 with d3w64. The wider network has more parameters even though it has fewer hidden layers. Counting layers alone therefore does not describe the storage or arithmetic budget. Likewise, adding a layer does not guarantee an accuracy gain on the held-out examples.

RECORD Which comparison changes width only? Which changes depth only? Which comparison changes both? State these distinctions before attributing an observed difference to a design choice.

Write a short conclusion using your quality and timing results. Include the parameter and MAC counts as explanations of model structure, and the measured latency as evidence from the device. Distinguish the theoretical estimate from the measured quantity.

3.8 Interpret the memory measurements

Use the saved `architecture_cost` results. Every model and trial ran in a fresh Python process, so one trial did not keep earlier interpreters in memory. The process loaded the same test-data mapping and a pool of 500 normalized images before the model. This keeps the workload preparation consistent across variants.

Saved metric	Meaning in this experiment
<code>file_kib</code>	The .tflite file size divided by 1,024.
<code>rss_before_model_mib</code>	Resident process memory after imports and common input preparation, before loading the model.
<code>rss_allocated_mib</code>	Resident process memory after constructing the interpreter and allocating tensors.
<code>rss_after_mib</code>	Resident process memory after warm-up and timed inference.
<code>peak_process_rss_mib</code>	Peak resident memory of the fresh process from its start through the final inference.

The summary reports `max_peak_process_rss_mib`, the largest process peak across the three trials. The tool reads Linux `VmRSS` and `VmHWM` from `/proc/self/status`. These kernel-reported counters include Python, libraries, data and runtime memory, and are not exact tensor accounting. Shared pages can contribute to RSS; do not sum process RSS values to estimate total physical RAM.

Architecture	File KiB	Maximum process peak MiB
d1w32		
d1w64		
d1w128		
d2w64		
d3w64		

Use the correct units: 1 KiB is 1,024 bytes and 1 MiB is 1,048,576 bytes. The increase from baseline RSS to allocated RSS can be informative, but it is affected by allocation and resident-page behaviour. Do not present this subtraction as an exact tensor-memory accounting.

INTERPRET A model file can be only a few hundred KiB while its Python process uses tens of MiB. Explain the difference. If two model files differ noticeably but process RSS is nearly equal, which costs may be dominating?

3.9 Brief comparison of numerical formats

Now keep the d1w64 architecture and its trained checkpoint fixed. Compare FP32, FP16 and INT8. Inspect both quantized artifacts before running them; the FP16 model still expects Float32 input, while the INT8 model expects encoded integer values.

RASPBERRY PI · Inspect the two alternative formats

```
python lab.py inspect --model d1w64_fp16
python lab.py inspect --model d1w64_int8
python lab.py predict --model d1w64_int8 --index 0
```

lab.py reads the tensor metadata and handles the encoding and decoding described in Chapter 2. You do not edit scale or zero-point manually. Evaluate the same 500 labelled examples, then measure the three artifacts under the same timing protocol.

RASPBERRY PI · Compare quality and execution cost

```
python lab.py evaluate --models formats \
  --limit 500 --run formats_quality
python lab.py benchmark --models formats \
  --warmup 20 --calls 500 --repeats 3 \
  --threads 1 --run formats_cost
python lab.py report --run formats_quality
python lab.py report --run formats_cost
```

Format	File KiB	Accuracy percent	Median ms	Peak MiB
FP32				
FP16				
INT8				

A file reduction is expected from reduced weight precision, but metadata prevents an exact 2× or 4× rule for the whole artifact. A smaller file may not execute faster on a particular CPU or use proportionally less process memory. Use the measurements rather than the format name to describe this board's result.

INTERPRET Calculate the INT8 file reduction relative to FP32. Report the accuracy change in percentage points and the latency speedup as FP32 median / INT8 median. State when a result is unchanged within the observed variation.

3.10 Inspect changed predictions

The `formats_quality` report also lists prediction agreement relative to `d1w64_fp32` and up to five changed examples. Two artifacts can have equal accuracy but disagree on individual images: some changes introduce an error and others correct one. Use the true label to distinguish these cases.

RASPBERRY PI · Read the comparison already saved

```
python lab.py report --run formats_quality
```

Choose an INT8 row from `changed_predictions.csv` or the report and replace INDEX below with its image index. If only FP16 predictions changed, use `d1w64_fp16` instead of `d1w64_int8` in the second command. If no classes changed, record zero. The full-test extension is a separate, explicitly labelled comparison.

RASPBERRY PI · Compare the same example

```
python lab.py predict --model d1w64_fp32 --index INDEX
python lab.py predict --model d1w64_int8 --index INDEX
```

Record	Your explanation
Image index and true class	
FP32 predicted class and top score	
INT8 predicted class and top score	
New error, correction, or different wrong answer?	

3.11 State the measurement conditions

The benchmark saves software versions, machine identity, thread count, timing boundary, warm-up count, model order and raw times. It also records temperature and `vcgencmd get_throttled` output when available. Missing telemetry is stored as null, which means unavailable, not zero temperature or an absence of throttling.

Inspect those records with the lecturer. Throttling flags can include historical events since boot as well as current conditions. Keep the same supply and cooling conditions, and record competing workloads. Repeating a trial is appropriate when you identify a reason; keep the original evidence and use a new run name.

LIMIT OF THIS EXPERIMENT The timed region excludes preprocessing and tensor copies. Process RSS includes the runtime and input pool. No energy consumption is measured. Do not infer energy savings or a guaranteed response deadline from these results.

3.12 Troubleshooting and recovery

Symptom	First action
No such file or directory	Confirm you are in ~/PL3 and that models and data were extracted with lab.py.
Cannot activate .venv or import LiteRT	Use <code>cd ~/PL3</code> and source <code>.venv/bin/activate</code> . If <code>.venv</code> is missing, complete section 3.2.
<code>python3 -m venv</code> fails	Confirm the <code>python3-venv</code> installation in 3.2 succeeded. Keep the error message and ask the lecturer for help.
No matching LiteRT distribution	Check <code>uname -m</code> and <code>python3 --version</code> . This package expects 64-bit Linux and a supported Python version.
Model or dataset hash mismatch	Download the official ZIP again in the Pi browser and extract a fresh copy. Do not benchmark a corrupted artifact.
Tensor contract error or REVIEW	Inspect the model ID, package release, runtime version and score differences. Ask the lecturer before benchmarking.
File exists or run folder exists	The tool protects earlier evidence. Choose a new run name such as <code>architecture_cost_r2</code> .
Accuracy differs from a classmate	Compare artifact hashes, subset size, preprocessing, runtime and exact predicted classes.
Timing varies or a larger model looks faster	Inspect trial ranges, CPU load, runtime configuration, temperature and throttling before explaining the result.
No changed classes for a quantized model	This is a valid observation for the evaluated subset. Scores may still differ.
Temperature or throttling is null	Telemetry was unavailable. Record that limitation and check the Pi tools with the lecturer.

The model tool provides command help without running experiments. Use the command-specific help to inspect available arguments. A failed command is not a reason to reinstall the whole system or change every package version.

RASPBERRY PI · Built-in help

```
python lab.py --help
python lab.py benchmark --help
```

KEEP THE EVIDENCE Preserve errors and the conditions that produced them. First fix the specific failed step, then resume from there. There is no need to repeat training in Colab to repair a download, extraction or environment problem.

3.13 Reflect on the results and resume later

Discuss what changed with width, depth and numerical representation. Use your tables and the saved results to support each explanation. Identify the board, test subset and measurement conditions whenever you compare numbers.

FOR PRACTICE AND DISCUSSION No upload, ZIP of results or formal submission is required. Keep your notes for your own learning. The CSV and JSON files remain in ~/PL3/results and the package check remains in ~/PL3/device_check.json.

Discuss before finishing	Use your observations to explain
Width and depth	A comparison that changes one architectural dimension at a time.
Storage and RAM	An explanation of why file size and process RSS differ.
Quantization	Same-checkpoint comparison, accuracy change and changed predictions.
Experimental limits	Board, sample size, timing scope and observed variation.

To return to your work in a new terminal, enter the PL3 folder and activate its environment again. The report commands below read the experiments you already saved; they do not run the measurements again.

RASPBERRY PI · Resume PL3 and inspect saved results

```
cd ~/PL3
source .venv/bin/activate
python lab.py report --run architecture_cost
python lab.py report --run formats_quality
```

You can close the terminal when finished. Reuse the resume block next time. To repeat an experiment deliberately, choose a new --run name so that the original results remain available.

Sources and further reading

The dataset images come from Fashion-MNIST by Zalando Research. The Moodle package includes its MIT licence and source attribution. The following official references explain the conversion and measurement interfaces used in this laboratory.

Fashion-MNIST dataset: github.com/zalando-research/fashion-mnist

LiteRT quantization: developers.google.com/edge/litert/conversion/tensorflow/quantization/post_training_quantization

Integer conversion: developers.google.com/edge/litert/conversion/tensorflow/quantization/post_training_integer_quant

Interpreter API: tensorflow.org/api_docs/python/tf/lite/Interpreter

Measurement: developers.google.com/edge/litert/models/measurement

Linux process memory: kernel.org/doc/html/latest/filesystems/proc.html